AI 스터디 WEEK3-4

숙명여자대학교 인공지능공학부 이나연 / 2026.02

<FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness>

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Abstract

문제점 : Transformer는 긴 sequence에 대하여 느리고 메모리 많이 소모함

원인 : self-attention의 시간 및 공간 복잡도가 sequence 길이에 quadratic하게 결정되기 때문.
이런 문제들을 다루기 위해 approximate attention 방법들이 시도되고 있음

⇒ 복잡도를 낮추려다가 model quality의 하락 / wall-clock speedup을 달성하지 못하기도 함

본 논문의 제안 : **FlashAttention**
⇒ IO-aware한 exact attention 알고리즘.
HBM과 SRAM 사이의 reads/writes 횟수를 줄이고, 속도가 빨라짐

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

1. Introduction

주장 : 성능 향상을 연구할 때 FLOPs reduction에만 집중하고 **memory access**는 무시하는 경향이 있음.
본 논문에서는 **IO-aware한 attention 알고리즘을 구현**하는 것이 중요한 포인트라고 주장함.

전략 : GPU의 각 계층의 빠르거나 느린 메모리들(on-chip SRAM, HBM)을 read, write하는 것을 유심히 살펴보자.

FlashAttention => memory access를 줄이는 알고리즘.

목표 : HBM에 attention 행렬을 read/write하는 것을 피하자. 최대한 SRAM에서 처리하자.

요구사항 : 1) softmax reduction을 전체 input으로 계산하지 않을 것.
2) 역전파를 위해 필요하지만 크기가 큰 intermediate attention 행렬을 저장하지 않을 것.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

기술
: 1) **tiling :** input을 block으로 쪼개서 계산
2) **recomputation** : softmax 정규화 factor를 저장하고, 역전파에 필요한 행렬은 on-chip으로 다시 계산하겠음. HBM에서 다시 읽어오는 것보다 빠름

=> attention operations를 하나의 GPU kernel로 합침. → 모든 연산을 하나의 커널(함수)로 합쳐서 한번에 실행하자.
FLOPs가 증가하더라도, 이 알고리즘이 **더 빠르고 메모리도 덜 사용함**

* d : head dimension / M : SRAM 크기
앞으로 다룰 내용
: 1) O(N^2*d^2*M^(-1))의 HBM access 증명
2) 모든 SRAM size에 대한 HBM access를 개선한 알고리즘으로, 하한선 제공
3) Block-sparse FlashAttention

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

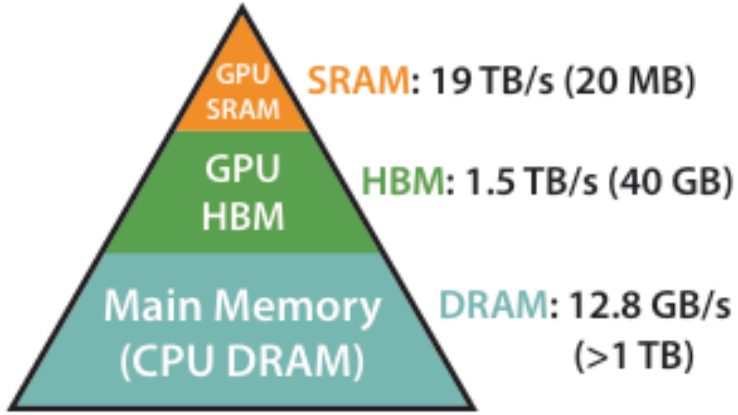
2. Background
2.1 Hardware Performace

GPU Memory Hierarchy
→ on-chip SRAM : 용량은 매우 작지만 매우 빠름.
HBM : 용량은 크지만 상대적으로 느림.

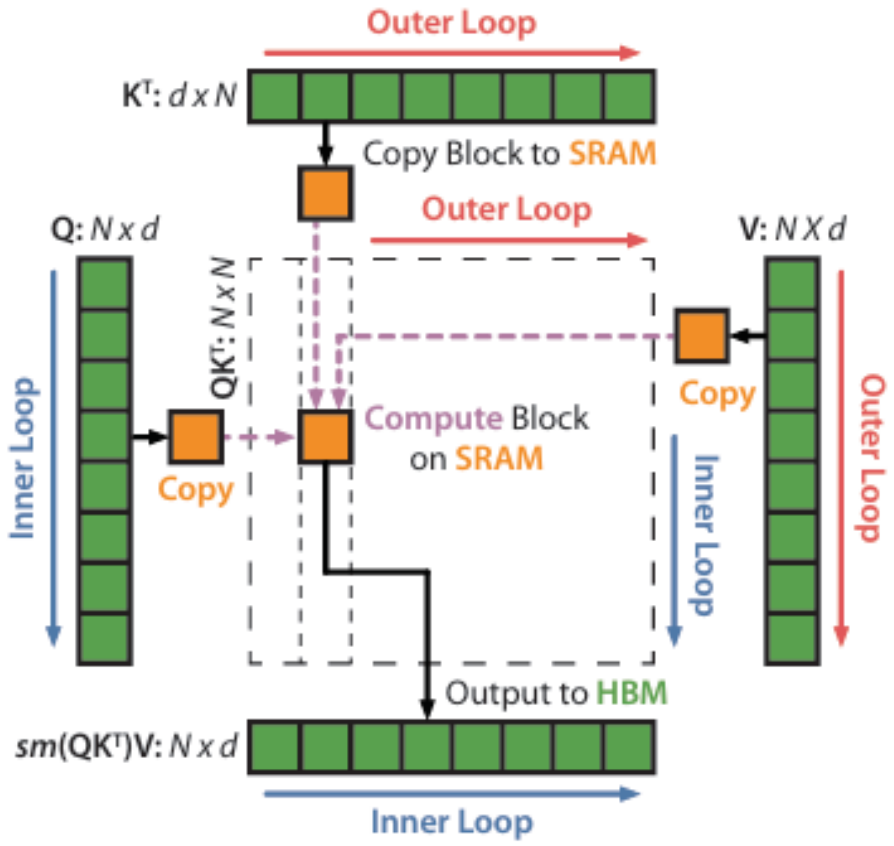
핵심 : memory 속도에 비해 compute(연산) 속도가 더 빨라짐
-> memory(HBM) access에서 implementation 병목 현상이 더 심해지고 있음.
따라서, 빠른 SRAM을 잘 사용하는게 더 중요해짐.

Execution Model
→ GPU는 수만개의 threads가 하나의 kernel(연산 단위)을 실행.
각 kernel 흐름 : input을 HBM에서 load → register/SRAM 이동 → 연산 → HBM에 output 작성

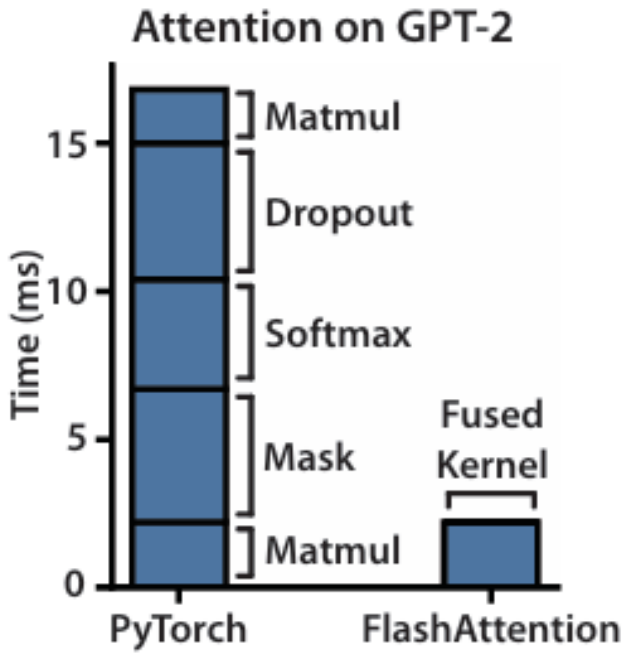
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness



Memory Hierarchy with Bandwidth & Memory Size



FlashAttention



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Performace characteristics
→ operation들을 compute-bound와 memory-bound로 나눌 수 있음.
arithmetic intensity(산술강도), memory access byte 당 arithmetic operations(FLOPs) 수를 기준으로 측정.

Compute-bound (연산 중심) : arithmetic operations 횟수로 시간이 결정. 산술 강도가 높아 GPU 연산 유닛이 계속 일하는 상태.
Memory-bound (메모리 중심) : memory access 횟수로 시간이 결정. 산술 강도가 낮아 연산을 금방 끝나는데 메모리 접근 시간이 긴 상태.

Kernel fusion
→ memory bound operations를 빠르게 하려면 kernel fusion을 고려해야함.
→ HBM에서 한번만 input을 load하고 중간 결과물은 SRAM 내에서 바로 사용하는 방법.

하지만, model train에서 역전파를 위해 중간 값을 HBM에 저장할 수 밖에 없음
-> kernel fusion의 효과가 감소함. FlashAttention은 이를 **Recomputation**으로 해결함.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

2.2 Standard Attention Implementation

input sequence : Q,K,V
attention output : O
sequence length : N
head dimension : d

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N\times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N\times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N\times d},$$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N\times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

S : 유사도 점수 행렬. Q와 K를 내적.
P : attention 가중치 행렬
O : 최종 출력 행렬

표준 attention은 S와 P를 HBM에 저장하고, O(N^2)의 메모리를 차지함.
보통 문장 길이(N)이 차원 수(d)보다 훨씬 큼. N >> d임. 그래서 **N^2**이 OOM의 주원인.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

2.2 Standard Attention Implementation

대부분 memory-bound operations이고 memory access 많음.
포인트는 HBM에 access하는 부분이 많다는 것. → 시간 병목 발생

* 참고
: FlashAttention은 수학적으로 표준 Attention과 완전히 동일한 결과를 내는 **Exact Attention**이면서도, 입출력(IO) 최적화를 통해 실행 속도를 획기적으로 개선함.

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

3. FlashAttention : Algorithm, Analysis , and Extensions
3.1 An Efficient Attention Algorithm With Tiling and Recomputation

목표 : HBM access 수를 N에 대해 sub-quadratic(이차 미만)으로 줄이자.
적용 기법 : tiling, recomputation

Tiling
→ attention을 block으로 계산하자.
Q,K,V를 block으로 쪼개서 느린 HBM →빠른 SRAM으로 로드함.
(블록별로 softmax를 그때그때 계산하고 바로 합쳐버려서 HBM에 저장할 필요가 없도록 하자)

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

벡터 x의 softmax연산 과정

$$m(x) := \max_i \; x_i, \quad f(x) := \left[e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}\right], \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

벡터 x(1), x(2)의 softmax 연산 과정

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)} \; x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)} \; x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)}) - m(x)} f(x^{(1)}) \quad e^{m(x^{(2)}) - m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)} \; x^{(2)}\right]) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)}), \quad \mathrm{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

⇒ SRAM 내에서 블록 별로 attention 계산
→ **나중에 하나로 합침** (정규화 계수 (nomalization factor)를 활용)

⇒ 전체 시퀀스를 한번에 다 보지 않고도,
<u>통계치 (m,l)을 업데이트하며 블록 단위로 정확한 softmax 값을 계산함.</u>

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Recomputation
→ 역전파를 위해 O(N^2) 크기의 중간 행렬 (S,P)를 HBM에 저장하지 않는게 목표.
→ 대신 출력 O와 softmax 통계치 m,l만 저장하고 역전파 때 SRAM에서 S,P를 다시 계산하자.
FLOPs는 늘어나겠지만, HBM 접근이 줄어들어 전체 속도는 더 빨라짐

Implementation details : Kernel fusion
→ Tiling으로 인해 전체 연산을 하나의 CUDA 커널로 구현하여 HBM 입출력을 최소화하는게 가능해짐

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Algorithm 1 살펴보기

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.

Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.

for $1 \le j \le T_c$ **do**

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  for $1 \le i \le T_r$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\tilde{m}_{ij} = \mathrm{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \mathrm{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.

    On chip, compute $m_i^{\mathrm{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\mathrm{new}} = e^{m_i - m_i^{\mathrm{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.

    Write $\mathbf{O}_i \leftarrow \mathrm{diag}(\ell_i^{\mathrm{new}})^{-1} (\mathrm{diag}(\ell_i) e^{m_i - m_i^{\mathrm{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.

    Write $\ell_i \leftarrow \ell_i^{\mathrm{new}}$, $m_i \leftarrow m_i^{\mathrm{new}}$ to HBM.

  end for

end for

Return $\mathbf{O}$.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Algorithm 1 살펴보기

1~2번 라인 : block 크기 설정 및 행렬 초기화.
*왜 M/4d인가 → SRAM에 Q,K,V,O 블록을 동시에 올리고 연산해야 하기 때문에,
각 블록의 크기(Bxd)를 고려해 4Bd ≤ M이 되는 B를 M/4d로 설정

3~4번 라인 : Q,K,V,O,l,m을 block으로 분리하기. Q,O,l,m은 Tr개 (행 개수) / K,V는 Tc개 (열 개수)
*l : 지수합 저장, softmax 분모 / m : max 저장 벡터

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.

2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

5~13번 라인
→ **for 문이 두 개 있음**. N x N을 Br x Bc조각으로 나눠서 모두 훑기 위함.
* K,V를 바깥 루프(j)에 둔 이유 : 상대적으로 크기가 큰 K,V 블록을 SRAM에 한 번 올려두고 여러 Q 블록과 연산하게 함으로써 HBM 읽기 횟수를 최소화하기 위한 것.

6,8번 : 데이터 HBM에서 SRAM으로 load해오기
9~11번 : 연산 과정
12~13번 : 값을 보정하고, HBM에 저장하는 과정.

5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \mathrm{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \mathrm{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\mathrm{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\mathrm{new}} = e^{m_i - m_i^{\mathrm{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \mathrm{diag}(\ell_i^{\mathrm{new}})^{-1} (\mathrm{diag}(\ell_i) e^{m_i - m_i^{\mathrm{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\mathrm{new}}$, $m_i \leftarrow m_i^{\mathrm{new}}$ to HBM.
14: **end for**
15: **end for**

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

3.2 Analysis: IO Complexity of FLASHATTENTION

복잡도 비교

→ standard attention : Big Theta(Nd + N^2) HBM 접근 요구 $\Theta(Nd + N^2)$

FlashAttention : Big Theta(N^2 d^2 M^(-1)) HBM 접근 요구 $\Theta(N^2 d^2 M^{-1})$

일반적으로, d^2 << M이니 FA가 더 작음

*어떻게 저런 복잡도를 가지게 되었는가?

(1) B는 M/d에 비례함 (d가 커질수록 올릴 수 있는 블록 크기인 B는 줄어듦)

(2) 바깥 루프의 Tc = N/B 은 Nd/M에 비례함

(3) 안쪽 루프의 Q에 대해 N*d만큼 필요

(4) Total : (Nd/M) * (N*d)

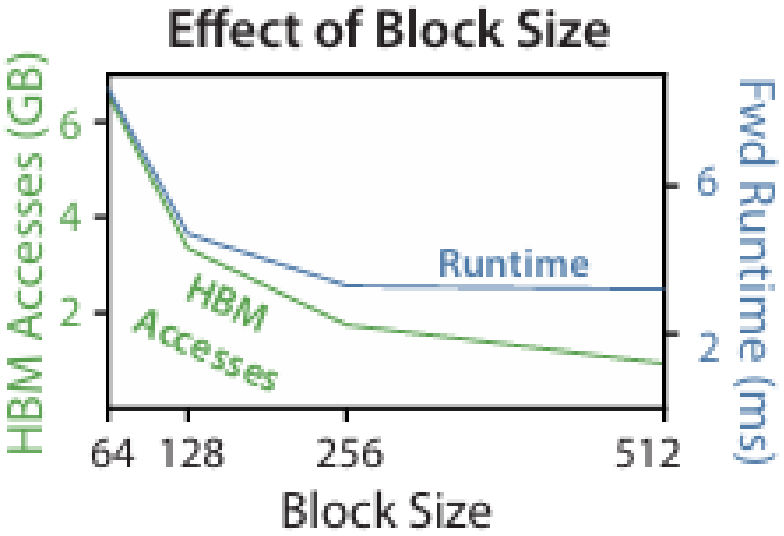
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

이 논문에서는 **Lower Bound**를 제시함
→ 어떤 알고리즘도, 모든 SRAM 크기에 대해 FA보다 더 나은 HBM 접근 횟수를 가질 수 없음.

즉, FlashAttention → Tiling&Recomputation 기법
→ 하한선에 도달한 최초의 최적 알고리즘

실험적 검증 : block 사이즈가 커질수록 HBM 접근이 줄어들며, 특정 지점까지는 runtime도 함께 감소함

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

3.3 Extension: Block-Sparse FLASHATTENTION
어떤 block은 아예 건너뛰어보자
→ 미리 정의된 **block sparsity(희소성) mask**를 사용하여,
zero/nonzero 블록 중 **nonzero 블록만** 계산하자.

Given inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ and a mask matrix $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$, we want to compute:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\tilde{\mathbf{M}}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some block sizes B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor$, $j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

IO복잡도가 희소성 비율인 **s**에 비례하여 개선됨.

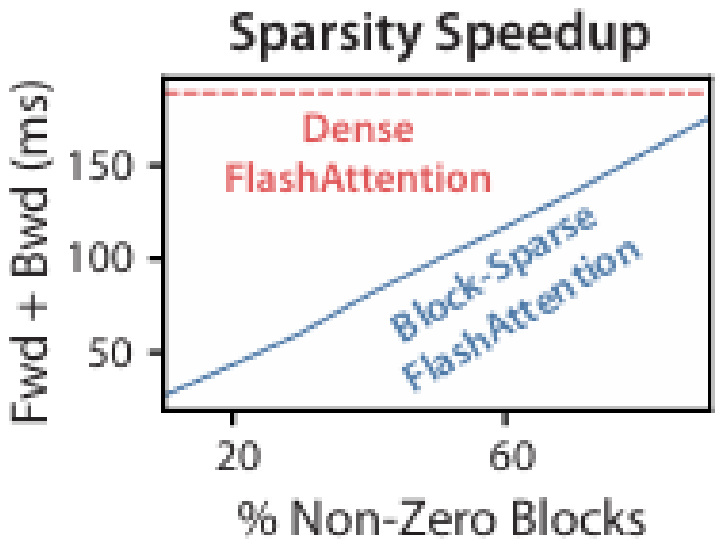
<u>Big Theta(Nd + N^2 d^2 M^{-1}s)에서</u>

$$\Theta(Nd + N^2 d^2 M^{-1} s)$$

Nd : 최소 기본 비용(한 번은 읽고 쓰는 비용)
(다른 수식에도 있긴 한데 여기서는 하한선으로 작용해서 더 중요함)
S : mask 행렬에서 1의 비율을 말함.

s는 주로 N^(-1/2) 또는 N^(-1)*logN이고, 이를 적용하면
N이 매우 클 때 아래와 같은 복잡도를 달성 가능

$$\Theta(N\sqrt{N}) \text{ or } \Theta(N \log N)$$



* 오른쪽 그래프에서
block-sparse FlashAttention이 runtime 속도가 더 빠르고,
비례적으로 증가함을 볼 수 있음.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

4. Experiments

4.1 Faster Models

BERT : MLPerft 1.1의 기존 최고 기록 (Nvidia)보다 **15%** 빠른 훈련 속도 기록 + 정확도 **72%**와 더불어 표에서 training time이 감소한 것을 확인할 수 있음

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention (ours)	17.4 ± 1.4

GPT-2 : HuggingFace 구현 대비 최대 3배, Megatron-LM 대비 **1.7배** 빠른 속도를 보임.
표에서 ppl은 유지되지만,
training time은 감소한걸 확인할 수 있음

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	2.7 days (3.5×)
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	6.9 days (3.0×)

Long-range Arena (LRA)
: 표준 Attention 대비 평균 **2.4배** 속도 향상 기록.
Block-sparse FlashAttention은 다른 방법들보다 더 빠른 것을 확인

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	2.8×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

4.2 Better Models with Longer Sequences

Language Modeling with Long Context

: context length를 4k로 늘려도 FlashAttention이 Megatron-LM의 1k 모델보다 **30%** 더 빠르고 성능(perplexity)도 **0.7** 더 우수함.
ppl은 개선되고 training time도 적은걸 확인할 수 있음

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	2.7 days (1.7×)
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	17.5	3.6 days (1.3×)

Long Document Classification

: 의료와 법률 관련 매우 긴 데이터인 MIMIC-III와 ECtHR에 대해서 성능 확인 가능

Table 5: Long Document performance (micro F_1) at different sequence lengths using FlashAttention.

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	57.1
ECtHR [6]	72.2	74.3	77.1	78.6	80.7	79.2

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

sequence 길이가 매우 긴 과제들인
Path-X와 Path-256 에서도,
Transformer 모델 최초로
무작위 성능보다 높은 유의미한 결과 달성.

* 이 두 과제는 긴 문맥을 이해해야 해결 가능한 흑백 이미지 기반 경로 찾기 문제.
기존의 transformer는
1) OOM이 발생하거나
2) 무작위로 작동했는데
=> FlashAttention은 해결함.

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	61.4	✗
Block-sparse FlashAttention	56.0	63.1

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

4.3 Benchmarking Attention

Runtime : runtime은 sequence 길이에 따라 quadratic하게 증가하지만, PyTorch 구현 대비 최대 3배 빠름. FlashAttention은 중간에 cross over 되는 지점이 있지만, block-sparse FlashAttention은 항상 좋음

Memory : 메모리 사용량이 sequence 길이에 따라 선형적으로 (linearly) 증가하고, 표준 attention 대비 최대 20배 효율적임. approximate(근사) attention에 대해서는 더 효과적임
Memory Footprint : 프로그램, 알고리즘이 실행하는 동안 실제로 점유하는 memory 총량

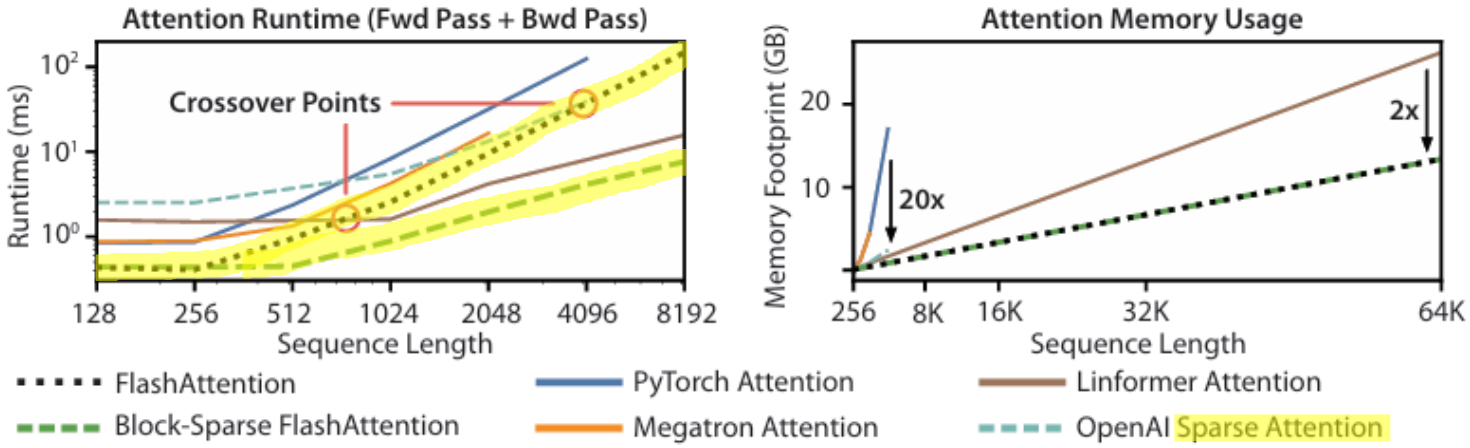


Figure 3: **Left:** runtime of forward pass + backward pass. **Right:** attention memory usage.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Limitaions and Future Directions

Compiling to CUDA
: 현재는 새로운 기능 넣으려면 저수준 언어인 CUDA로 직접 커널을 짜야 함.

IO-Aware Deep Learning
: Attention 외 다른 딥러닝에도 이러한 IO-aware한 접근 방식 적용할 필요가 있음

Multi-GPU
: 단일 GPU를 넘어, 여러 GPU간 데이터 전송까지 고려한 분석이 미래 과제임.

감사합니다 :0